

動物クラス設計書（継承）

1. 概要

本設計書は、動物を表す `Animal` クラスを基底クラスとし、犬（Dog）と猫（Cat）を継承によって実装する構造を示す。ポリモーフィズムを利用し、共通のインターフェース `speak()` を通じて異なる鳴き声を返す仕組みを確認する。

2. クラス設計

2-1. 親クラス Animal

(1) 役割

動物の基本情報（名前）と、デフォルトの鳴き声を提供する基底クラス。

(2) 属性

属性名	型	説明
name	str	動物の名前

(3) メソッド

メソッド名	説明
speak()	デフォルトの鳴き声 "... " を返す

2-2. 子クラス Dog

(1) 役割

犬を表すクラス。Animal を継承する。

(2) メソッド

メソッド名	説明
speak()	犬の鳴き声 "ワン" を返す

2-3. 子クラス Cat

(1) 役割

猫を表すクラス。Animal を継承する。

(2) メソッド

メソッド名	説明
<code>speak()</code>	猫の鳴き声 "ニャー" を返す

3. 動作確認仕様

(1) インスタンス生成

以下の 4 つの動物を生成し、リストに格納する。

- `Dog("ポチ")`
- `Dog("シロ")`
- `Cat("タマ")`
- `Cat("ミケ")`

(2) for 文による処理

リスト内の各インスタンスに対して
`animal.speak()` を呼び出し、
名前と鳴き声を表示する。

(3) ポリモーフィズムの確認

- 変数 `animal` は `Animal` 型として扱われるが、
実際には `Dog` または `Cat` の `speak()` が呼び出される。
- 継承とメソッドオーバーライドによる動作の違いを確認できる。

4. 動作結果

```
ポチ：ワン  
シロ：ワン  
タマ：ニャー  
ミケ：ニャー
```

5. まとめ

本設計では、`Animal` クラスを基底として `Dog` と `Cat` が継承し、
`speak()` メソッドをオーバーライドすることで、
同じメソッド呼び出しでも異なる動作を実現している。
これはオブジェクト指向の基本概念である「ポリモーフィズム」を示す典型的な例である。